

Configurer et utiliser GitHub

GitHub est une plateforme essentielle pour les développeurs. Elle permet de stocker, partager et collaborer sur des projets de code. En utilisant **GitHub**, tu peux bénéficier du contrôle de version fourni par Git et exploiter les outils de collaboration pour améliorer la qualité et l'efficacité de tes projets.

Pourquoi utiliser **GitHub** ?

- **Contrôle de version** : **Git** te permet de suivre toutes les modifications apportées à ton code. Chaque modification est enregistrée sous forme de "commit", ce qui te permet de revenir à une version précédente si nécessaire. Tu peux également voir qui a modifié quoi et pourquoi.
- **Collaboration** : **GitHub** facilite le travail en équipe en permettant à plusieurs développeurs de travailler sur le même projet simultanément. Tu peux gérer les contributions via des "pull requests", réviser et commenter le code, et intégrer les changements de manière contrôlée.
- **Open Source** : **GitHub** héberge des millions de projets open source auxquels tu peux contribuer. C'est un excellent moyen de collaborer avec des développeurs du monde entier et d'apprendre en travaillant sur des projets réels.
- **Intégration continue (CI)** : **GitHub** s'intègre avec des outils de CI/CD (Intégration Continue / Déploiement Continu) comme Travis CI, CircleCI, et **GitHub** Actions, qui automatisent les tests et le déploiement, garantissant que ton code est toujours dans un état fonctionnel.

Configuration de GitHub

1. Créer un compte GitHub

Commence par créer un compte sur [GitHub](#). Ce compte te permettra de créer et de gérer des dépôts de code, de collaborer avec d'autres utilisateurs et de profiter de toutes les fonctionnalités offertes par la plateforme.

2. Installer Git

Git est l'outil qui te permet de gérer les versions de ton code. Pour l'utiliser, il faut d'abord l'installer sur ton ordinateur. Voici comment procéder selon ton système d'exploitation :

- **Windows** : Télécharge le programme d'installation depuis le [site officiel de Git](#). Suis les instructions pour l'installer. Pendant l'installation, tu pourras configurer certaines options ; prend le temps de bien les lire et les comprendre. Ajoute ensuite **Git** à ton *PATH des variables d'environnement*, ce qui te permettra de l'utiliser depuis n'importe quel terminal.
- **macOS** : Ouvre l'application Terminal et tape `git --version`. Si **Git** n'est pas installé, macOS te proposera de l'installer automatiquement. Sinon, tu peux l'installer via [Homebrew](#) en tapant `brew install git`.
- **Linux** : Sur la plupart des distributions Linux, **Git** peut être installé via le gestionnaire de paquets. Par exemple, sur Ubuntu ou Debian, tu utiliseras `sudo apt-get install git`. Pour Fedora, ce serait `sudo dnf install git`.

Une fois **Git** installé, tu peux vérifier l'installation en tapant `git --version` dans ton terminal. Cela devrait afficher la version de **Git** installée.

3. Configurer Git

Avant de commencer à utiliser **Git**, tu dois le configurer avec ton nom et ton email. Ces informations seront associées à chaque commit que tu fais, ce qui permet d'identifier qui a effectué quelles modifications.

```
git config --global user.name "TonNom"
git config --global user.email "tonemail@example.com"
```

- `--global` : Cette option applique cette configuration à l'ensemble de ton environnement **Git**. Tu n'auras à le faire qu'une seule fois sur chaque machine.
- `user.name` et `user.email` : Ce sont les informations d'identification que **Git** va utiliser pour enregistrer les auteurs des commits.

Tu peux vérifier que tout est configuré correctement en utilisant la commande :

```
git config --list
```

Cela affichera toutes les configurations actuelles de **Git**, y compris ton nom et ton email.

Git est un logiciel de gestion de versions qui permet de suivre les modifications apportées à des fichiers, notamment dans le cadre de projets de développement.

GitHub, en revanche, est une plateforme en ligne qui utilise **Git** pour héberger des projets, faciliter la collaboration entre développeurs et offrir des outils de gestion de projets, comme le suivi des *issues* et les *pull requests*.

4. Créer une clé SSH

L'utilisation d'une **clé SSH** permet une connexion sécurisée entre ton ordinateur et **GitHub**. Cela évite d'avoir à entrer ton mot de passe à chaque interaction avec **GitHub**, comme les pushes ou les pulls.

1. Vérifier si tu as déjà une clé SSH :

Avant de créer une nouvelle **clé SSH**, vérifie si tu en as déjà une. Pour cela, tape la commande suivante dans ton terminal :

```
ls -al ~/.ssh
```

Cette commande liste les fichiers du répertoire `~/.ssh`. Si tu vois des fichiers comme `id_rsa` ou `id_ed25519`, c'est que tu as déjà une clé SSH. Les fichiers `id_rsa.pub` ou `id_ed25519.pub` sont les clés publiques associées. Si tu n'as pas de fichier `.ssh` ou que le répertoire est vide, tu devras générer une nouvelle clé.

2. Générer une nouvelle clé SSH :

Si tu n'as pas de **clé SSH**, ou si tu veux en créer une nouvelle, utilise cette commande :

```
ssh-keygen -t ed25519 -C "tonemail@example.com"
```

- **-t ed25519** : Spécifie l'algorithme de cryptage. **ed25519** est recommandé car il est plus sûr et plus rapide que **rsa**. Si ton système ne supporte pas **ed25519**, utilise plutôt :

```
ssh-keygen -t rsa -b 4096 -C "tonemail@example.com"
```

- **-C** : Ajoute un commentaire (généralement ton email) à la clé, pour l'identifier plus facilement.

Lorsque tu es invité à entrer un chemin pour sauvegarder la clé, appuie sur "Entrée" pour utiliser l'emplacement par défaut (**~/.ssh/id_ed25519** ou **~/.ssh/id_rsa**). Tu peux aussi choisir de protéger la clé avec une passphrase, ce qui ajoute une couche supplémentaire de sécurité.

3. Ajouter la clé SSH à l'agent SSH :

L'*agent SSH* gère tes *clés SSH*. Pour qu'il prenne en charge ta nouvelle clé, utilise ces commandes :

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

- **eval "\$(ssh-agent -s)"** : Lance l'agent SSH.
- **ssh-add ~/.ssh/id_ed25519** : Ajoute ta **clé SSH** privée à l'agent, pour qu'elle soit utilisée automatiquement par Git.

4. Trouver l'empreinte de la clé SSH :

L'empreinte de ta **clé SSH** est une signature unique qui te permet de vérifier son intégrité. Pour la trouver, utilise la commande suivante :

```
ssh-keygen -lf ~/.ssh/id_ed25519.pub
```

Cela retournera quelque chose comme :

```
256 SHA256:G1l/vNjfw0kVnDGHruZA5KmmbibepdEhvB7RYnrAVa0 tonemail@example.com  
(ED25519)
```

- **256** : La taille en bits de la clé.

- **SHA256:...** : L'empreinte de la clé. Elle te permet de vérifier que c'est bien la clé que tu veux utiliser.
- **tonemail@example.com** : Le commentaire que tu as ajouté (ton email ici).

5. Ajouter ta clé SSH à GitHub :

Maintenant que ta **clé SSH** est prête, tu dois l'ajouter à ton compte **GitHub** pour pouvoir l'utiliser.

1. Copie la clé publique dans ton presse-papiers avec cette commande :

- Pour macOS :

```
cat ~/.ssh/id_ed25519.pub | pbcopy
```

- Pour Linux :

```
cat ~/.ssh/id_ed25519.pub | xclip
```

Si tu utilises Windows, ouvre le fichier **id_ed25519.pub** avec un éditeur de texte et copie son contenu manuellement.

2. Connecte-toi à ton compte **GitHub** et va dans [les paramètres SSH](#).

3. Clique sur "**New SSH key**", donne un titre à ta clé (par exemple "Mon ordinateur portable"), puis colle ta clé publique dans le champ "**Key**". Enfin, clique sur "Add SSH key".

6. Tester la connexion :

Pour t'assurer que tout est correctement configuré, teste ta connexion à **GitHub** en utilisant :

```
ssh -T git@github.com
```

Si tout fonctionne bien, tu devrais voir un message de bienvenue comme :

```
Hi username! You've successfully authenticated, but GitHub does not provide shell access.
```

Cela signifie que ta **clé SSH** est correctement configurée et que tu peux maintenant interagir de manière sécurisée avec **GitHub**.

Utilisation de GitHub

Une fois Git et SSH configurés, tu peux commencer à utiliser **GitHub** pour gérer tes projets.

1. Cloner un dépôt

Le clonage d'un dépôt te permet de récupérer une copie complète du projet sur ton ordinateur. Pour cloner un dépôt, utilise la commande suivante :

```
git clone git@github.com:utilisateur/nom-du-depot.git
```

- **utilisateur/nom-du-depot.git** : Remplace **utilisateur** par le nom d'utilisateur du propriétaire du dépôt et **nom-du-depot** par le nom du dépôt que tu veux cloner.

Le dépôt sera téléchargé dans un dossier local, et tu pourras commencer à travailler dessus.

2. Travailler sur ton projet

Après avoir modifié des fichiers, il est temps d'enregistrer tes modifications dans le dépôt **Git** local.

1. **Ajouter les modifications** : Utilise **git add .** pour ajouter toutes les modifications au prochain commit. Si tu veux ajouter un fichier spécifique, utilise **git add nom-du-fichier**.

- **git add .** : Ajoute toutes les modifications dans le répertoire actuel au prochain commit.
- **git add <nom-du-fichier>** : Ajoute un fichier spécifique.

2. **Vérifier l'état du projet** : Avant de committer, il est recommandé de vérifier l'état de ton dépôt avec :

```
git status
```

- **git status** : Affiche les fichiers modifiés, ajoutés ou supprimés et indique les étapes nécessaires pour les committer.

3. **Créer un commit** : Un commit enregistre tes modifications avec un message descriptif :

```
git commit -m "Description des modifications"
```

- **git commit -m "<message>"** : Crée un commit avec un message de description des changements.

Sois précis dans tes messages de commit pour faciliter la compréhension des modifications.

4. **Envoyer les modifications à GitHub** : Une fois tes modifications commit, envoie-les sur le dépôt GitHub avec :

```
git push origin main
```

- **git push <remote> <branche>** : Envoie les commits de ta branche locale vers la branche correspondante sur GitHub.
- **origin** : C'est le nom par défaut de la télécommande qui pointe vers ton dépôt GitHub.
- **main** : La branche principale du projet.

3. Collaborer avec d'autres

Lorsque d'autres personnes contribuent au même projet, tu dois mettre à jour ta copie locale pour intégrer leurs modifications. Utilise :

```
git pull origin main
```

- **git pull <remote> <branche>** : Récupère les dernières modifications depuis GitHub et les intègre dans ta copie locale.

Si des conflits surviennent lors du merge, Git t'informera et tu devras les résoudre manuellement avant de finaliser l'intégration.

4. Gérer les branches

Les branches sont essentielles pour travailler sur différentes fonctionnalités ou corrections sans affecter la branche principale.

1. **Lister les branches** : Pour voir toutes les branches disponibles dans ton dépôt local, utilise :

```
git branch
```

- **git branch** : Liste toutes les branches locales dans ton dépôt.

2. **Créer une nouvelle branche** : Crée une branche pour développer une nouvelle fonctionnalité ou corriger un bug :

```
git checkout -b nouvelle-branche
```

- **git checkout -b <nouvelle-branche>** : Crée et bascule immédiatement sur la nouvelle branche.

3. **Changer de branche** : Pour revenir à une autre branche (par exemple **main**) :

```
git checkout main
```

- **git checkout <branche>** : Change de branche en basculant sur une branche existante.

4. **Fusionner une branche** : Une fois que ta nouvelle fonctionnalité est prête, tu peux la fusionner avec la branche principale :

```
git checkout main  
git merge nouvelle-branche
```

- **git merge <branche>** : Fusionne une branche spécifiée avec la branche actuelle.

Si tout se passe bien, la branche sera fusionnée. En cas de conflit, Git te demandera de les résoudre.

5. **Supprimer une branche** : Après avoir fusionné une branche et vérifié qu'elle n'est plus nécessaire, tu peux la supprimer :

```
git branch -d nouvelle-branche
```

- **git branch -d <branche>** : Supprime une branche locale.

Si la branche n'a pas été fusionnée avec une autre branche, Git te demandera une confirmation ou refusera la suppression. Tu peux forcer la suppression avec **git branch -D <branche>**.

5. Afficher l'historique des commits

Pour voir l'historique des commits de ton projet, utilise :

```
git log
```

- **git log** : Affiche la liste des commits effectués, y compris les messages de commit, les auteurs et les dates.

Tu peux aussi afficher un historique plus condensé avec des options comme **--oneline** :

```
git log --oneline
```

- **git log --oneline** : Affiche l'historique des commits sous forme condensée, avec un identifiant abrégé et le message du commit.

6. Revenir à une version précédente

Il arrive parfois que tu souhaites revenir à une version antérieure de ton projet. Pour cela, tu peux utiliser la commande **git checkout** ou **git reset**.

1. **Revenir à une version précédente temporairement** :

```
git checkout <commit>
```

- **git checkout <commit>** : Bascule temporairement sur un commit spécifique sans modifier l'historique. Le commit peut être identifié par son hash (identifiant).

2. Revenir de manière permanente :

```
git reset --hard <commit>
```

- **git reset --hard <commit>** : Réinitialise l'état du dépôt au commit spécifié, supprimant toutes les modifications ultérieures.

Attention : Cette commande supprime toutes les modifications effectuées après le commit spécifié et peut être irréversible.

7. Annuler un commit

Si tu as fait une erreur dans un commit récent, tu peux l'annuler avec **git revert** ou **git reset**.

1. Annuler le dernier commit sans supprimer les modifications :

```
git reset --soft HEAD~1
```

- **git reset --soft HEAD~1** : Annule le dernier commit, mais conserve les modifications dans l'index et le répertoire de travail.

2. Annuler le dernier commit et les modifications :

```
git reset --hard HEAD~1
```

- **git reset --hard HEAD~1** : Annule le dernier commit et supprime toutes les modifications apportées.

8. Staging des modifications pour plus tard

Si tu travailles sur quelque chose et que tu dois temporairement mettre tes modifications de côté pour travailler sur autre chose, tu peux utiliser la commande **git stash** :

```
git stash
```

- **git stash** : Met de côté les modifications en cours sans les committer, permettant de revenir à un état propre.

Pour récupérer les modifications stashed, utilise :

```
git stash apply
```

- **git stash apply** : Récupère les modifications mises de côté avec **git stash**.

9. Suivre un fichier ou annuler le suivi

Si tu as un fichier que tu ne veux plus suivre avec Git (par exemple un fichier de configuration généré), tu peux le retirer de l'index avec :

```
git rm --cached <fichier>
```

- **git rm --cached <fichier>** : Retire un fichier de l'index sans le supprimer du répertoire de travail.

Conclusion

En configurant GitHub avec une clé SSH, tu sécurises tes interactions et simplifies ton flux de travail. Que tu gères un projet personnel ou que tu collaboras avec une équipe, GitHub offre des outils puissants pour suivre, gérer et partager ton code efficacement. Continue à explorer les fonctionnalités de GitHub et améliore tes projets grâce à ses nombreuses possibilités. Bonne programmation !